

Keycloak Credential Rotation

Testing & Verification Runbook

ANR-2553 — STIG Compliance | ESO-Native Dual-Credential Rotation (PostgreSQL keycloak user + Master Realm Admin)

1. Architecture Summary

The keycloak rotation case is an **ESO-native** dual-credential rotation that rotates both the PostgreSQL *keycloak* user password (K8s Secret: `keycloak-pgpass`) and the Keycloak master realm admin password (K8s Secret: `keycloak-admin-password`) in a single pipeline run. The script **never generates a password** — External Secrets Operator's `password-gen` Password generator is the single source of truth. The script triggers ESO to regenerate each Secret, reads the new value, then propagates it to PostgreSQL (via `ALTER USER`) and Keycloak (via `kcadm.sh`).

The rotation follows a strict DB-before-app ordering: the database password is rotated and the Keycloak deployment is restarted **before** the admin password is touched. Both credentials are back-vaulted to Azure Key Vault immediately after their respective rotations, with automatic rollback if back-vaulting fails.

Phase	Steps	Source of Truth	Action
Pre-rotation	1	K8s Secrets	Capture old DB + admin passwords for rollback
Phase 1: DB	2-8	ESO password-gen → K8s Secret	Force-sync → poll → ALTER USER → rollout → verify → back-vault
Phase 2: Admin	9-13	ESO password-gen → K8s Secret	Force-sync → poll → kcadm.sh → verify token → back-vault

2. Prerequisites Checklist (Verify Before First Run)

Run each command below in your WDSF terminal. All checks must pass before the rotation can be triggered via the pipeline. Record the output of each — this evidence goes into the PR.

2.1 ESO Version (must be **>= v0.10.0** for force-sync)

```
kubectl get deployment -n external-secrets external-secrets \
-o jsonpath='{.spec.template.spec.containers[0].image}'
```

Expected: image tag contains v0.10.0 or higher. If lower, force-sync annotation will not work and the ESO controller must be upgraded first.

2.2 Password Generator Exists and Is Healthy

```
kubectl get password password-gen -n auth -o yaml
```

Expected: kind: Password, spec.length: 20, spec.symbolCharacters: '!@#._+:=,?:'. Record the full spec for the PR.

2.3 Both ExternalSecrets Exist and Point to password-gen

```
kubectl get externalsecret -n auth -o yaml
```

Expected: two ExternalSecrets (keycloak-pgpass, keycloak-admin-password), both with refreshInterval: 0s, both with dataFrom.sourceRef.generatorRef.name: password-gen.

2.4 Both K8s Secrets Are ESO-Owned

```
kubectl get secret -n auth keycloak-pgpass keycloak-admin-password -o yaml \
| grep -E "ownerReferences|reconcile.external-secrets.io/managed"
```

Expected: each Secret has ownerReferences pointing to its ExternalSecret AND the label reconcile.external-secrets.io/managed: "true". If either is missing, ESO does not own the Secret and the rotation will fail.

2.5 Pods Are Healthy and Reachable

```
kubectl get pods -n auth -l "app.kubernetes.io/name=keycloak-db"
kubectl get pods -n auth -l "app.kubernetes.io/name=keycloak"
```

Expected: keycloak-db-0 is Running (StatefulSet), keycloak pods are Running and Ready. Record pod names and ages.

2.6 DB User Verification

```
DBPOD=$(kubectl get pods -n auth -l "app.kubernetes.io/name=keycloak-db" -o
jsonpath="{.items[0].metadata.name}")
kubectl exec -i $DBPOD -n auth -- psql -U keycloak -c "SELECT current_user;"
```

Expected: output shows "keycloak". This confirms the DB user is keycloak (superuser), not postgres or bn_keycloak.

2.7 Key Vault Reachability

```
Get-AzKeyVaultSecret -VaultName <KeyVaultName> -Name "keycloak-pgpass"
Get-AzKeyVaultSecret -VaultName <KeyVaultName> -Name "keycloak-admin-password"
```

Expected: both return without error. The pipeline service connection must have Set permission (not just Get) for back-vaulting to work.

2.8 ArgoCD Sync Paused

```
# Pause ArgoCD self-heal on the keycloak application to prevent
# ExternalSecret/Secret reverts mid-rotation:
kubectl patch application keycloak-standalone-core -n argocd \
--type='json' \
-p='[{"op": "replace", "path": "/spec/syncPolicy/automated/selfHeal", "value": false}]'
```

Expected: selfHeal set to false. Resume after rotation completes (Section 5, step 8). If selfHeal is left true, ArgoCD may revert ESO's Secret regeneration mid-rotation, causing drift.

2.9 Keycloak Image Has curl

```
KCPOD=$(kubectl get pods -n auth -l "app.kubernetes.io/name=keycloak" -o
jsonpath="{.items[0].metadata.name}")
kubectl exec $KCPOD -n auth -- which curl
```

Expected: returns a path like /usr/bin/curl. If curl is not found, the health check in step 6 of the rotation will fail. Bitnami Keycloak images include curl; Quarkus native images may not.

3. Pre-Rotation State Capture

Run these commands **immediately before** triggering the pipeline. The captured values are your baseline for post-rotation diff comparison and your rollback safety net. Record all outputs.

3.1 Capture Current Secret Values (Base64)

```
$dbBefore = kubectl get secret keycloak-pgpass -n auth \
-o jsonpath="{.data.keycloak-pgpass}"
$admBefore = kubectl get secret keycloak-admin-password -n auth \
-o jsonpath="{.data.keycloak-admin-password}"
Write-Host "DB (base64): $dbBefore"
Write-Host "Admin (base64): $admBefore"
```

Record both base64 strings. After rotation, you will compare these against the post-rotation values to confirm the Secrets actually changed.

3.2 Verify Current Admin Login Works

```
$adminPlain = [System.Text.Encoding]::UTF8.GetString(
[System.Convert]::FromBase64String($admBefore))
kubectl port-forward deployment/keycloak -n auth 8080:8080 &

# In another terminal:
curl -s -X POST "http://localhost:8080/realms/master/protocol/openid-connect/token" \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "username=admin&password=$adminPlain&grant_type=password&client_id=admin-cli"

# Expected: HTTP 200 with "access_token" in JSON response
```

Record the HTTP status code. This is your proof that the old admin password works before rotation. After rotation, this same request should return HTTP 401.

3.3 Record Pod Names and Ages

```
kubectl get pods -n auth -l "app.kubernetes.io/name=keycloak-db" -o wide
kubectl get pods -n auth -l "app.kubernetes.io/name=keycloak" -o wide
```

Record pod names, ages, and restart counts. After rotation, the Keycloak pods should have age < 5 minutes (freshly restarted) and restart count unchanged (no crash loops).

4. Execute the Rotation

Trigger the rotation via the established pipeline. The keycloak case drops into the existing `switch ($toolLower)` block in `Rotate-Credentials.ps1` and runs automatically when

TARGET_TOOL=keycloak is passed.

Parameter	Value	Notes
TF_Module	infrastructure	Must be infrastructure (has the Terraform outputs)
TF_Action	apply	Rotation only runs after a successful apply
ROTATE_CREDENTIALS	true	Enables the K8s Credential Rotation stage
TARGET_TOOL	keycloak	Selects the keycloak switch case

The pipeline will execute all 13 steps automatically. Expected runtime: 4-8 minutes (dominated by Keycloak rollout and master realm initialization). Monitor the pipeline console output for the step markers [1/13] through [13/13].

5. Post-Rotation Verification

Run these commands **after** the pipeline reports success. Every check must pass before the rotation is considered complete. Record all outputs for the PR.

5.1 Both Secrets Have New Values

```
$dbAfter = kubectl get secret keycloak-pgpass -n auth \
-o jsonpath="{.data.keycloak-pgpass}"
$admAfter = kubectl get secret keycloak-admin-password -n auth \
-o jsonpath="{.data.keycloak-admin-password}"
if ($dbAfter -eq $dbBefore) { Write-Error "DB Secret did NOT rotate!" }
if ($admAfter -eq $admBefore) { Write-Error "Admin Secret did NOT rotate!" }
Write-Host "Both Secrets rotated successfully"
```

Expected: "Both Secrets rotated successfully". If either Secret value matches the pre-rotation value, ESO did not regenerate it — investigate ESO controller logs before proceeding.

5.2 Keycloak Pods Ready and Stable (60s)

```
kubectl get pods -n auth -l "app.kubernetes.io/name=keycloak"
Start-Sleep 60
kubectl get pods -n auth -l "app.kubernetes.io/name=keycloak"
```

Expected: all pods Running and Ready, age < 5 min, RESTARTS column unchanged between the two checks (no crash loops). Record both outputs.

5.3 No DB Auth Failures in Logs

```
$logs = kubectl logs -n auth -l "app.kubernetes.io/name=keycloak" --tail=200
if ($logs -match "28P01|password authentication
failed|HikariPool.*fail|Agroal.*fail|connection.*refused") {
Write-Error "DB auth failures detected: $($matches[0])"
} else {
Write-Host "DB pool healthy" -ForegroundColor Green
}
```

Expected: "DB pool healthy". If failures are detected, the ALTER USER may have succeeded but the Keycloak connection pool is still using the old password — a rollout restart should resolve it, but investigate if it persists.

5.4 New Admin Password Issues Token

```
$newAdminPlain = [System.Text.Encoding]::UTF8.GetString(
[System.Convert]::FromBase64String($admAfter))
kubectl port-forward deployment/keycloak -n auth 8081:8080 &

curl -s -X POST "http://localhost:8081/realms/master/protocol/openid-connect/token" \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "username=admin&password=$newAdminPlain&grant_type=password&client_id=admin-cli"

# Expected: HTTP 200 with "access_token" in response
```

Record the full HTTP response. The presence of "access_token" confirms the new admin password works end-to-end.

5.5 Old Admin Password Is Rejected

```
curl -s -X POST "http://localhost:8081/realms/master/protocol/openid-connect/token" \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "username=admin&password=$adminPlain&grant_type=password&client_id=admin-cli"

# Expected: HTTP 401 with "invalid_grant" or "Unauthorized"
```

Record the HTTP status code. This confirms the old password is no longer valid — the rotation took effect.

5.6 Azure Key Vault Has New Values

```
$dbKv = (Get-AzKeyVaultSecret -VaultName <KeyVaultName> -Name
"keycloak-pgpass").SecretValueText
$admKv = (Get-AzKeyVaultSecret -VaultName <KeyVaultName> -Name
"keycloak-admin-password").SecretValueText
if ($dbKv -ne $newDbPlain) { Write-Error "KV DB password mismatch!" }
if ($admKv -ne $newAdminPlain) { Write-Error "KV admin password mismatch!" }
Write-Host "Key Vault back-vault verified" -ForegroundColor Green
```

Expected: "Key Vault back-vault verified". Record the Key Vault secret version IDs (the id field from the Get-AzKeyVaultSecret output) for the PR evidence.

5.7 ESO Ownership Intact (No Drift)

```
kubectl get secret keycloak-pgpass -n auth -o yaml | Select-String
"ownerReferences"
kubectl get secret keycloak-admin-password -n auth -o yaml | Select-String
"ownerReferences"
kubectl get secret keycloak-pgpass -n auth -o yaml \
| Select-String "reconcile.external-secrets.io/managed"
kubectl get secret keycloak-admin-password -n auth -o yaml \
| Select-String "reconcile.external-secrets.io/managed"
```

Expected: both Secrets show ownerReferences pointing to their ExternalSecret AND the managed label. This confirms ESO still owns the Secrets (no manual delete/create drift was introduced by the rotation).

5.8 Resume ArgoCD Self-Heal

```
kubectl patch application keycloak-standalone-core -n argocd \
--type='json' \
-p='[{"op":"replace","path":"/spec/syncPolicy/automated/selfHeal","value":true}]'
```

Expected: selfHeal set back to true. Record the timestamp. This was paused in Section 2.8 and must be resumed after all verification checks pass.

6. PR Evidence Checklist

Gather the following artifacts for the PR. These are the proof points a reviewer needs to confirm the rotation is safe, correct, and STIG-compliant.

- ☐ **Pre-rotation secret hashes** — SHA-256 of the old DB and admin password plaintext (not the plaintext itself). Computed via: `[Security.Cryptography.SHA256]::Create().ComputeHash([Text.Encoding]::UTF8.GetBytes($oldDbPass)) | ForEach-Object {$_.ToString('x2')}`
- ☐ **Post-rotation secret hashes** — SHA-256 of the new DB and admin password plaintext. Must differ from pre-rotation hashes.
- ☐ **Pre-rotation admin token test** — HTTP 200 + access_token present (Section 3.2)
- ☐ **Post-rotation admin token test** — HTTP 200 + access_token present (Section 5.4)
- ☐ **Old admin password rejection** — HTTP 401 (Section 5.5)
- ☐ **Keycloak pod health** — Two `kubectl get pods` outputs 60s apart showing no restarts (Section 5.2)
- ☐ **DB pool health** — Log scan showing no 28P01/FATAL/password auth failures (Section 5.3)
- ☐ **Key Vault secret version IDs** — The `id` field from `Set-AzKeyVaultSecret` output for both `keycloak-pgpass` and `keycloak-admin-password` (Section 5.6)
- ☐ **ESO ownership proof** — `kubectl get secret ... -o yaml` showing `ownerReferences` + `managed` label for both Secrets (Section 5.7)
- ☐ **ArgoCD self-heal pause/resume timestamps** — When `selfHeal` was set to false (Section 2.8) and when it was restored to true (Section 5.8)
- ☐ **Pipeline console output** — Full log showing steps [1/13] through [13/13] with no errors. Save as an attachment to the PR.
- ☐ **password-gen spec** — `kubectl get password password-gen -n auth -o yaml` output confirming `length: 20` and `symbolCharacters: '!@#.-_+:=,?:'` (Section 2.2)

7. Failure Modes & Rollback Matrix

The rotation script has built-in rollback for the two most critical failure points (DB back-vault failure and admin back-vault failure). For other failure modes, manual intervention is required. The table below

documents each failure point, its symptom, and the recovery action.

Failure Point	Symptom	Recovery Action
ESO force-sync doesn't trigger	Script throws "ESO did not regenerate within 60s"	Check ESO controller logs. Verify ESO >= v0.10.0. Verify password-gen exists. Retry.
ALTER USER fails (psql non-zero)	Script throws "ALTER USER failed"	DB catalog unchanged, K8s Secret has new value. Re-run ALTER USER manually with the new value from the Secret, then rollout restart Keycloak.
Keycloak rollout fails (CrashLoopBackOff)	Script throws on rollout status timeout	Check Keycloak logs. If DB connection issue, the ALTER may have failed silently. Verify DB password matches Secret, then re-rollout.
DB back-vault fails	Script catches, rolls back ALTER, restarts Keycloak	Automatic rollback. Keycloak is on old DB password. Fix Key Vault connectivity, then re-run the full rotation.
kcadm.sh fails after 5 retries	Script throws "kcadm.sh failed after 5 attempts"	DB rotated successfully, admin password in Keycloak DB is OLD, K8s admin Secret has NEW value. Manual: use Keycloak Admin UI (port-forward) with old admin password to set it to the K8s Secret's new value.
Admin back-vault fails	Script catches, rolls back kcadm.sh, throws	Automatic rollback. Admin password in Keycloak DB is back to old value. Fix Key Vault, then re-run admin rotation only.

Key principle: If any rollback itself fails (e.g., ALTER USER rollback throws), the script will throw and halt. At that point you need manual DBA intervention. Do not attempt to re-run the pipeline until the state is reconciled manually.